

VICTORIA UNIVERSITY OF WELLINGTON
Te Whare Wananga o te Upoko o te Ika a Maui



School of Engineering and Computer Science
Te Kura Mātai Pūkaha, Pūrorohiko

PO Box 600
Wellington
New Zealand

Tel: +64 4 463 5341
Fax: +64 4 463 5045
Internet: office@ecs.vuw.ac.nz

Unsupervised Analysis of Rat Ultrasonic Vocalisations

Siôn Grabham-Madden

Supervisor(s): Bastiaan Kleijn, Bart Ellenbroek

Submitted in partial fulfilment of the requirements for
Bachelor of Engineering with Honours

Abstract

Rats are highly social animals who regularly make use of Ultrasonic Vocalisations (USVs), beyond the range of human hearing, to convey information such as whether the rat is experiencing an aversive or affective state, or to make maternal separation calls in the case of infants. USVs provide insights into the rat's state in a non-invasive fashion useful in multiple fields of study, however the categorisation and classification of complex affective USVs still largely relies on manually identified and categorised schemes of discrete symbols. Manual categorisation presents the possibility for human bias in the identification of relevant features, so methods to automatically produce a syllable repertoire, such as the existing solution implemented in DeepSqueak, are desirable. However the use of manually defined features in DeepSqueak presents some similar potential limitations. This project uses unsupervised learning techniques based on processes used in human speech phoneme classification, on unlabeled audio data in order to provide a system capable of producing fully unsupervised and automatically produced USV syllable categorisation schemes, to remove human bias and labour from the call analysis process in future research.

Table of Contents

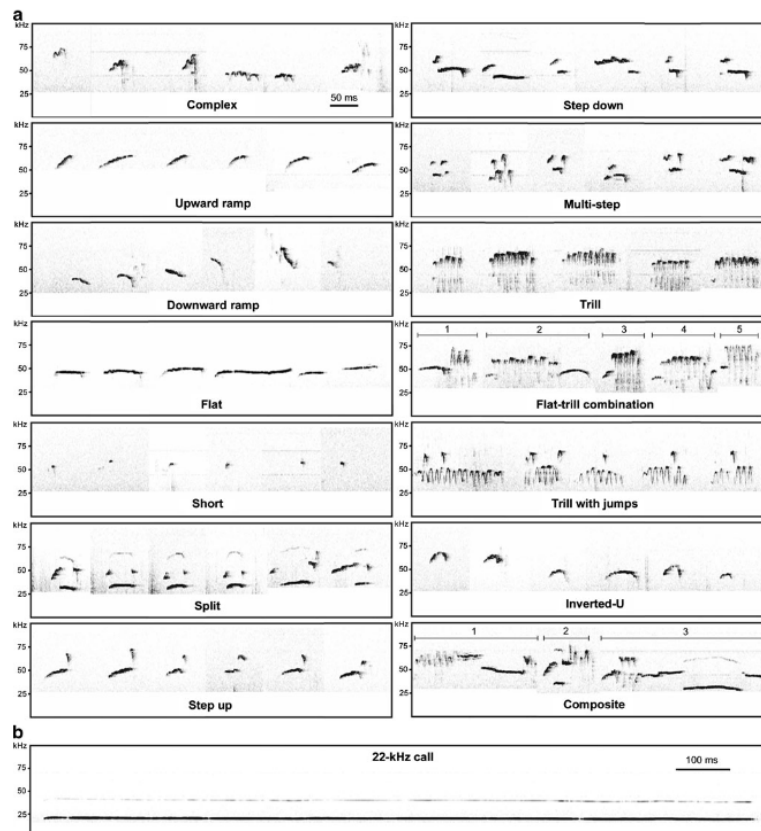
1. Introductory Summary	1
2. Background and Related Work	5
3. Initial Solution	10
4. Revised Design	15
5. Final Implementation	22
6. Experiments and Evaluation	27
7. Conclusions and Future Work	31

1. Introductory Summary

Rats are highly social animals who regularly vocally communicate, however almost all vocalisations by rats are ultrasonic, inaudible to human ears (Brudzinsky, 2009). These Ultrasonic Vocalisations (USVs) are used at different frequencies in many social contexts. These different frequency bands are quite broad but generally center around three different average points; USVs at around 22 kHz in aversive contexts such as defeat in an aggressive encounter or the presence of a predator, around 50 kHz in affective or appetitive contexts including various social stimuli and food cues, and around 40 kHz in maternal separation calls by isolated infants (Takahashi et al., 2010).

Researchers have analyzed basic aspects of USVs, including their regularity, frequency, and modulation, for some time. Better understanding patterns of rat vocalisation has provided a non-invasive technique to gain insights in multiple fields, including neurodevelopment and pharmacology (Simola, 2015; Sungur et al., 2015). USVs provide information as to the vocalising rat's state, relevant in studies including determining whether a rat displays aversive or appetitive reactions to a drug for addiction studies, or diagnosing a rat with developmental issues and/or autism.

Even basic metrics have led to significant developments in better understanding rats, but there is room for much deeper analysis. In particular, the 50 kHz range affective call type contains rich complexity and variety which has opened the door to establishing categorisation schemes of subtypes or "syllables" of call, which may lead to further understanding of meaning. However the categorisation and classification of affective USVs and their meaning, including the analysis of the syllables or units of sound generally accepted to be present, still largely relies on a manually identified scheme of fourteen categories or sub-types (Wright et al., 2010).



The authors of (Wright et al., 2010) call for automatic categorisation solutions, primarily to reduce significant workloads involved. (Coffey et al., 2019) argues that manual categorisation also leads to injections of human bias, and proposes DeepSqueak, a software system with a solution that clusters call spectrogram images based on simple automatically extracted features to produce its own category schemes for the specific purposes of researchers. However, the basic feature set used in DeepSqueak may poorly capture the complexity of affective calls, particularly any cross-segment contextual information. (Goffinet et al., 2021) highlights the potential limitations of the use of handpicked or manually defined features, particularly in possibly missing dimensions of variability.

Behavioural neurogenetics researchers at Victoria University of Wellington expressed an interest in a new software system, able to take unlabeled USV recording data, and automatically produce call categorisation schemes in an unsupervised fashion, without using manual features and considering criticisms against current clustering results. This would go further towards elimination of potential human biases in the process of the construction of sets of syllable units. It should be designed in such a way as to allow the categorisation schemes developed within it to be easily applied to new arbitrary USV data later. This would allow automatic symbolic representation of USV calls. Effectively, researchers could easily generate “transcripts” of a rat’s calls, significantly aiding the process of actually assigning calls to categories in practice for future downstream tasks.

The utility of being able to generate and save multiple different categorisation schemes in the system to apply to later data is significant. Such a feature grants users the ability to gradually improve and refine a categorisation model to improve its fit to actual behaviour in the future, which will be key in the research goal of answering the question raised in (Goffinet et al., 2021) as to how well USVs can actually be modeled as distinct clusters. Additionally, it allows users to account for the fact that not all rats vocalize the same way. The acoustic features of rat USVs are not necessarily universal to all rats, just as not all phonemes are universally present across all human languages, though the extent of this difference is still an area of active research.

(Wright et al., 2010) calls for studies to generalize their findings beyond the adult male Long-Evans strain rats they used exclusively. Notable differences in USV characteristics have been discovered both across sex and strain variations when studied (Graham et al., 2009; Lenell et al., 2021). In fact, it is possible to use affective and aversive USV characteristics to automatically distinguish between two different strains of rat (Mittal et al., 2018). Accounting for this lack of universal generalizability, by allowing strain and/or sex-specific categorisation schemes to be generated and easily used in study, will make a system significantly more robust for the future as understanding in this area grows.

With the right preprocessing of the audio data, including modifying sample rates and significantly reducing noise, it is possible to use a deep learning model called wav2vec2.0 for our feature extraction. This is an open-sourced Facebook AI Research model, originally for the generation of general representations of human speech audio. Convolutional neural networks extract 1024 features per 20ms time-step of the audio data, and a transformer network then contextualizes these features across the full signal (Baevski et al., 2020). This results in a sequence of highly featured contextualized representations across the span of the audio file.

This representation of a USV call signal addresses the feature selection concerns of (Goffinet et al., 2021) and accounts for the possibility of inter-segment context. Signal representations can then be processed into high-dimensional segment tensors representing individual USV calls, through a series of feature processing steps that primarily rely on clustering timesteps based on similar features and mean-pooling adjacent steps of the same clusters, analogous to the process used for human speech phoneme segmentation in wav2vec-u (Baevski et al., 2021). With a sufficiently large and informative dataset, the resulting representations can then be utilized to generate new, automatic categorisation schemes. To also do this in an unsupervised way, clustering based on call segment features is used.

The HDBSCAN algorithm is selected for clustering in the final implementation. It extends DBSCAN density-based clustering over varied parameter values to optimize for the best stability, which allows the algorithm to identify clusters of varied densities as well as varied shapes (McInnes et al., 2017). Using HDBSCAN, a set of clusters can be calculated that function as our call categories. This achieves our primary goal, and the hyperparameters of the clustering algorithm serve as additional valves for researchers to tweak if necessary in generating schemes useful to their purposes.

HDBSCAN also allows new data to be processed after generating a primary clustering result, using soft clustering to predict the probabilities a new data point fits into each existing cluster. Processing new USV call audio in the same way the main data was in order to create additional call representation segments, these segments can be used as an input to HDBSCAN soft clustering predictions. The highest-probability cluster assignments for each segment are recorded as well as their millisecond lengths, effectively outputting an ordered transcript of the categories of each call. This gives us our symbolic representation.

This means we have a fully automated pipeline available to researchers through a handful of commands, to generate transcripts of USV call audio into any given categorization scheme generated by the system. Additionally, tools are provided to easily generate new schemes that can be used in the same way, providing the tools to find better and/or more fit for purpose sets of communication subunits for arbitrary research goals.

The particular goals and structure of this project present difficulties in evaluating its success quantitatively. With a goal of explicitly avoiding experimenter-driven errors and human bias in the definition of a correct result, there are no “true” call assignments to measure against, meaning there is no equivalent to Phoneme Error Rate or Word Error Rate in human speech processing, where a specific correct answer is known and sought for. Creating an artificial model “language” similar to the calls of rats with clearly defined true categories of USV equivalents could allow for a quantitative performance measurement of category identification, though doing so represents an independently complex task which did not fit in the timeline of this project.

While evaluating a specific categorisation scheme quantitatively is not possible, comparing transcript outputs with spectrographic visualizations of the same signal does allow us to evaluate the effectiveness of our USV call detection and isolation. It is also possible to qualitatively judge how well a categorisation fits visual expectations for researchers, based on an indication of spectrographic patterns within categories or a lack thereof. Evaluating the categorisation scheme generated experimentally at the end of this project visually revealed a healthy performance in call separation, but patterns of call category assignment were not considered effective enough for lab use.

The implemented system achieves all primary goals, but there is still significant potential to improve. Processing speed could be much faster. Automatic noise suppression is imperfect, leaving a potential gap where human intervention may still be necessary. And the qualitative evaluation of the experimental categorisation scheme generated in testing indicates that refinements may be necessary to system processes to produce schemes at the level of quality needed for real use in future studies. The system is complete and functional, but improvements or additions in several areas could elevate it from an interesting experimental approach, to a practical solution seeing use in real USV analysis research going forward. Potential avenues of improvement are discussed in detail in the final section.

2. Background and Related Work

Manual Call Analysis

Ultrasonic Vocalizations for communication in rats have been identified and studied for decades (Sales, 1972), but categorization of rat USVs for more detailed analysis has been a long process of gradually expanding repertoires of types of call. Category repertoires of three or four categories of 50kHz range USVs were established in the 1990s (Kaltwasser, 1990; White et al., 1990; Vivian and Miczek, 1993), but after identifying richer diversity in call spectrograms, and following the identification of as many as ten USV subtypes in a study of mouse pups (Scattoni et al., 2008), (Wright et al., 2010) would establish the broadly accepted categorization scheme for rat USVs.

Adult male rats of the Long-Evans strain were administered saline solutions and amphetamines in separate tests, and recorded both individually and in pairs with a cage mate, in order to gather over 20,000 USV call spectrograms for researchers to visually analyse manually. Surgical devocalizations were also performed in order to confirm that the calls identified were legitimate based on their abolition in subsequent recordings following the surgery.

The resulting categorisation scheme, which can be seen in Figure 1, consists of fourteen categories of 50 kHz range affective call types, as well as a single call category for 22 kHz range aversive calls which the authors found were infrequent and indistinct enough to disregard for the rest of their analysis. Most of the fourteen affective call categories define a particular shape of spectrogram, while two combine trills with other shapes, and one category, the “Composite” category, consists of single calls that combine a number of different spectrographic shapes.

Further, about 1% of calls were classified as “miscellaneous” for not clearly fitting into any single category, and 7% of calls were found to be “unclassifiable” for being spectrographically unclear. Of calls that were classified into categories, however, inter-rater reliability and intra-rater reliability were found to be 96% and 97% respectively, indicating strong consistency in the assignment of calls to these categories once they had been defined.

The rate of occurrence of different call categories was far from even, with half of all affective calls falling into three of the fourteen categories. Trills alone accounted for 29%, flat-trill combinations 16%, and flat calls 14%. Comparatively, the least common category, split calls, accounted for only 0.5% of all calls. This is not necessarily an indication of any flaw in the categorisation scheme, differences in rate of occurrence are to be expected, though a visual comparison of the similarity of the exemplar calls of the Split category and the Multi-Step category might indicate that the separation of these categories could be considered arbitrary, given the statistical backing of their rarity.

In any case, that affective call categories might vary by orders of magnitude in terms of rate of occurrence is a key insight for future solutions. They need to be designed in such a way that categories of very different sizes can be identified from the same dataset, if they are indeed present.

While it represented a significant leap forward from previous more simple solutions, (Wright et al., 2010) does not claim to be a comprehensive or complete system for categorizing rat USV calls. That all of its data is drawn from male rats of the same strain and around the same level of adult development is acknowledged, and whether the categorisation scheme created generalizes to female rats and rats of other strains and ages can not be determined without significant additional study. In fact, generalizability is highlighted as a goal for future investigation, in order to more fully understand the results found and how they fit into the reality of rat communication.

Along with generalizability, and calling for research to begin into behavioural significance of different categories, the paper calls for automation as an essential part of future full solutions. Manual categorization is described as highly labor intensive, and the possibility of observer bias in categorization as well as the possibility of uncovering further complexities not addressed in the present classification of categories through mathematical modeling are both mentioned.

The next steps from (Wright et al., 2010) are clear, in that automation of USV call type analysis is essential to attain full confidence in any solution, and to ease the process of actual categorisation to the point of being practical. Further, we know this should ideally be done in a way that can handle data from a more general set of rats, and is robust in analyzing call types which may form very different shapes and sizes of cluster. However, simply automatically identifying USV calls from background noise is a challenging problem already, let alone the generation of category schemes and then automatic symbolic representation. There are a multitude of potential avenues that could be pursued to solve this string of problems, several of which have already seen exploration, as detailed below.

Previous Automatic Call Analysis Solutions

Mirroring the gradual development of manual USV analysis studies, solutions for the automation of the distinct but related challenges of rat USV call identification, and subsequently call categorization, have developed and expanded over time.

In the closely related field of mouse USV analysis, (Holy & Guo, 2005) developed an automated algorithm that identified the “syllables” of ultrasonic vocalization “songs” in male mice. Sonograms were generated from audio data, using thresholding to remove noise and cutting to the relevant frequency range, before saving resulting sonograms as sparse complex matrices. These sparse sonograms would then be processed to calculate three time-varying parameters, being median-filtered over ten milliseconds.

These three parameters included mean frequency, spectral purity, and spectral discontinuity, the latter two of which were more complexly mathematically derived and helped pick out syllables. Where mean frequency exceeded 35 kHz, spectral purity exceeded 0.25, and spectral discontinuity was below 1 for a period of 5 ms, a syllable was automatically identified. Syllables less than 30 ms apart were then merged, to account for the splitting of faint vocalisations by the noise removal process.

Evaluation of this algorithm showed a strong performance, with a less than 5% rate of false negatives (that is, failing to identify a true USV syllable) compared to human inspection of the same spectrograms. Additionally, false positives (identifying a USV syllable where there is none) were extremely rare, only two being found in several hours of recording. In fact, the authors note that many vocalizations were identified by the algorithm that had initially been missed by human inspection, only to be later verified.

This highly effective result indicated that automated techniques can perform very well in USV call identification, and in fact may surpass manual processes with the right implementation. (Holy & Guo, 2005) also made the significance of noise reduction as a part of automatic analysis clear, showing the importance of isolating the relevant signal while working around the difficulty of not suppressing faint sections of actual USV calls.

Tools to more deeply analyse USVs automatically, including discovery of new syllable repertoires, would come later. (Van Segbroeck et al., 2017) introduced MUPET, or Mouse Ultrasonic Profile ExTraction, a MATLAB-based software capable of syllable repertoire construction (synonymous with call category scheme generation), producing inter-syllable metrics to compare, and time-stamping syllables from a USV call input as a form of symbolic representation/transcription for downstream research tasks like matching call types to particular behaviours.

MUPET is an openly available tool that takes audio files of any length and a sampling rate of at least 90 kHz. Preprocessing of audio data includes a high-pass filter with a corner frequency at 25kHz to attenuate out sounds below the ultrasonic range, before generating sonograms optimized for a balance between time and frequency resolution. Subsequently, noise reduction processes calculate a “noise floor spectrum” and subtract this over the frequency range. How harsh this noise reduction is can be modified by the user, to balance between less noise getting through and suppressing faint calls.

Syllable detection is performed by identifying the areas where spectral energy in the ultrasonic range is above a given noise threshold, taking the timestamps where energy passes above and below that threshold as the start and end of each syllable. A gammatone filterbank is applied to create simpler feature representations that capture key spectral features. Representations are clustered using k-means clustering of the cosine distance between vectorised images. The user can pick out “repertoire units” of noise to remove, before final repertoires of varying sizes are created, and the user can then select an optimal repertoire size based on modeling scores and goodness-of-fit measures.

MUPET is designed for mice first, but the authors outline the steps necessary for using the system with other rodent species, or mice of different ages and/or strains, mostly a process of designing a purpose-built featurebank to use instead of the provided one. This tool represented another step forward in automation, but it relies on user decision-making at several key points. Additionally, (Coffey et al., 2019) found that using the best-fit metrics of MUPET to determine repertoire size typically led to large repertoires that would be difficult to practically map to behaviour. These repertoires were also not possible to save, for future application to new recordings.

DeepSqueak was introduced in (Coffey et al., 2019) in part to address these concerns. It makes use of deep learning tools, in particular a regional convolutional neural network, to more fully automate the process of USV detection. The provided R-CNNs (one for mice, one for short rat USVs, one for 22 kHz rat USVs, and one general purpose network) were trained on hundreds of manually isolated USV sonograms and thousands of USVs isolated by initial small networks, for the task of identifying USV calls in a sonogram image. The training is optimized for high “recall”, the ratio of detected true USVs to total true USVs in a given file.

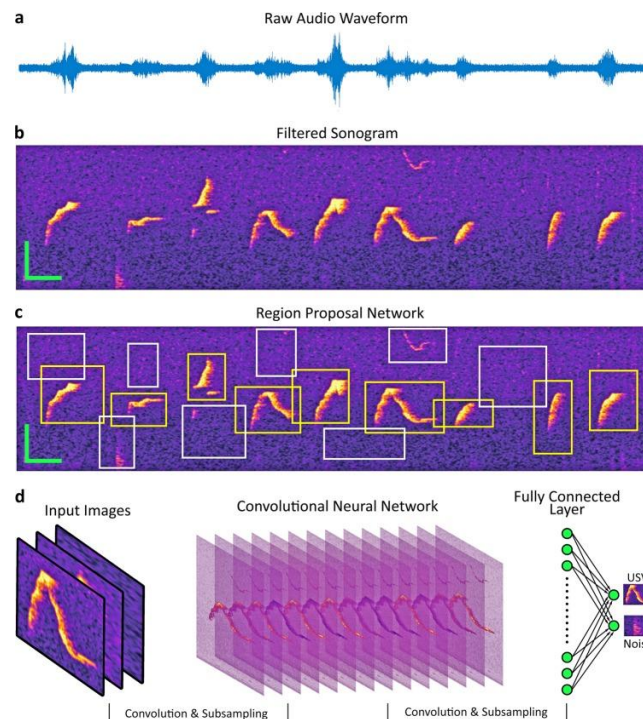


Figure 2: R-CNN USV Detection in DeepSqueak, from (Coffey et al., 2019).

The USV detection R-CNN in DeepSqueak is very sensitive, tuned to ensure faint vocalizations are picked up, at a trade off of more frequent false positives. Following this, a robust set of methods for removing false positives due to noise are provided to compensate. Confidence scores for each detection are generated by the R-CNN, and detections can be batch rejected by user-generated thresholds of this score, by spectral power, or by call tonality. Additionally, a secondary detection network is provided with the tool that is trained to detect noise and separate it from true USVs. The user can also label noise and USVs in their audio to train their own model of the denoising network, if the provided one is not performant.

After sufficient post-hoc denoising leaves the user primarily with genuine USV detections, the spectrogram of each is processed through a contour detection algorithm that identifies the highest amplitude frequency at each time point, in order to extract a USV “shape”. To perform syllable clustering in DeepSqueak, features are then extracted from this contour information. These features are the call frequency and shape (a derivative of the contour) at each time point, and the full duration of the contour. Frequency, shape, and duration are each equally weighted by default, and serve as the input to the clustering stage.

DeepSqueak’s primary clustering algorithm is k-means, as with MUPET. However, while the user can still define a number of clusters to sort USVs into, DeepSqueak is also capable of automatically calculating a most appropriate set of clusters using the elbow method. This method optimizes for the least squared error, a measure of how well-fitted or not the clustering is, without overfitting to unreasonable numbers of clusters, one of the issues identified with MUPET in (Coffey et al., 2019). The elbow method functions by calculating the summed squared error at different numbers of clusters (which naturally decreases as clusters increase) and finding the point along the resulting curve where straight lines along all the points left and right of it best fit the curve.

This implementation of the elbow method for k-means clustering allowed DeepSqueak to offer automatic and unsupervised categorisation of rat USVs, a major step forward in the field. The tool refines the clustering techniques used in MUPET, which have also seen success in unsupervised analysis of other animal call patterns such as the whistles of dolphins (Kershenbaum et al., 2013). DeepSqueak added to previous work establishing clustering as a legitimate technique for unsupervised USV classification, particularly by offering a solution that would automatically determine a most appropriate number of clusters. This tool comprised another advance towards full automation and unsupervision, and the removal of potential human bias from USV categorisations.

DeepSqueak is not immune to experimenter-driven error, however. It still relies in part on the user to distinguish noise from true USVs in the form of false positive detections, a leak of decision-making by humans into the process. More significantly, the structure of its USV representations is in and of itself manually defined, with human researchers having handpicked the small number of domain-specific features. This strategy reduces computational complexity, but presents a number of issues including the potential to poorly represent or entirely miss dimensions of genuine variability, and the tendency of handpicked features to statistically correlate (Goffinet et al., 2021).

The structure of our USV representations, and how they are generated, is perhaps the most significant area in which improvements can be made on DeepSqueak to continue to advance the drive towards the full elimination of experimenter-driven error potential. It will also be necessary to account for another difficult question raised for the field in (Goffinet et al., 2021), that clustering of USVs is in general less effective than the same process on some other forms of animal communication. A UMAP two dimensional projection of mouse USV features indicated much more contiguous and overlapping data than in clearly separable syllables of a bird song, for example, though such projections necessarily involve loss of information to dimensionally compress the data. Goodness of clustering metrics for USVs were generally found to be poor, but above that of gaussian noise, and (Goffinet et al., 2021) ultimately puts forward that clustering may not best represent USV variability.

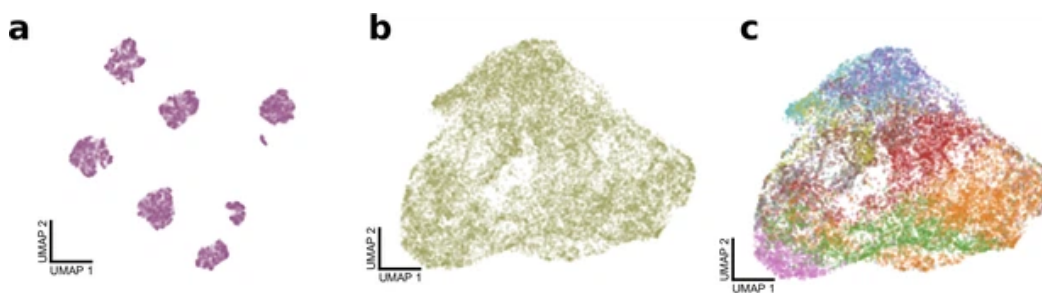


Figure 3: UMAP Projection of Zebra Finch calls (A) compared to mouse USVs (B). C represents colored indication of MUPET clustering assignments of calls, from (Goffinet et al., 2021).

How effectively clustering can serve as a classification technique for USVs remains an open question academically, and new clustering algorithms may significantly impact performance in the future. It is outside the scope of this project to conclusively determine whether USVs fit into discrete syllabic categories through clustering, or whether data ultimately supports a continuum of differing meaning. However, by providing new tools that allow for the analysis of automatically generated categorisation schemes simply and with less necessary accounting for human error, future studies in this area will be significantly eased. Consequently, this project will continue to explore improvements in the use of clustering as an unsupervised method for USV analysis, and allow later researchers to use the tool to draw their own conclusions based on the effectiveness of their results in practical use.

In 2019, Facebook AI Research introduced wav2vec, a convolutional neural network model that made use of unsupervised pre-training, to generate representations of raw audio for downstream speech recognition tasks (Schneider et al., 2019). The following year, wav2vec2.0 would be developed, improving on the original model's performance and making use of newer machine learning innovations including a transformer structure to allow each data point of the time sequence to be fully contextualized to the entire sequence (Baevski et al., 2020).

Another year on, progress in this area continued with the development of wav2vec-U, a system making use of wav2vec2.0 models for feature extraction, as an input to a new feature processing design that allows for unsupervised speech recognition using a phoneme segmentation solution (Baevski et al., 2021). Wav2vec models were identified as a novel and interesting approach to feature extraction on USV audio, as their design addresses the handpicked feature concerns of (Goffinet et al., 2021) and other than model fine-tuning no labeled data is necessary. Rather than attempting to directly build off of previous designs in DeepSqueak and MUPET, this project will explore the use of wav2vec models as a feature extraction solution.

Key Insights

USV analysis through categorisation is a developing field, and successive studies and tools have iteratively improved on their predecessors towards the goal of full unsupervised processing, from raw audio data to final calls and their categories. Key challenges include the complicated and varied structure of the data, the universal presence and confounding nature of noise, and the difficulty posed by the goal of removing human decision-making from the process as much as possible.

Selecting true USVs out from noise and silence is a key part of the puzzle for a solution to these challenges. Following this, the tool must be capable of extracting representative features from the data without manual definition of the nature of those features. Wav2vec models offer a new and potentially effective approach to both of these requirements. Clustering must then be implemented, accounting for weak separation of data in past studies, to allow for unsupervised classification. Finally, it will be essential to provide useful outputs to the user to actually inspect the effectiveness of their results, easing the future of USV categorisation analysis significantly.

3. Initial Solution

Wav2vec was selected for use in this project to extract features from USV data in an unsupervised fashion without requiring data labels. There are several kinds of wav2vec model, but the original as outlined in (Schneider et al., 2019) was chosen for the initial solution as some advice on the Facebook AI Research github indicated that obtaining the feature representation data for downstream tasks unrelated to human speech recognition would be most simple with these models. This also presented complications in adapting systems and techniques identified as potentially useful from academic papers that had been designed for later versions of the model, a tradeoff considered acceptable in the early stages of project design.

Wav2Vec is built on a Convolutional Neural Network design. These are analogous to traditional artificial neural networks that can reduce the number of required parameters and so support larger models for complex tasks such as facial recognition in image classification. CNNs are named so because they use convolution to connect inputs into a convolution layer. Applying a fixed matrix of weights called the filter across only a local region of the input data drastically reduces the connections required when compared to a traditional ANN, and by effectively sliding a filter window across the input, features can be detected and recognized regardless of their spatial position. Further, adding additional layers of convolution which might have different configurations means different kinds of features can be extracted at different levels from the same input data (Albawi et al., 2017).

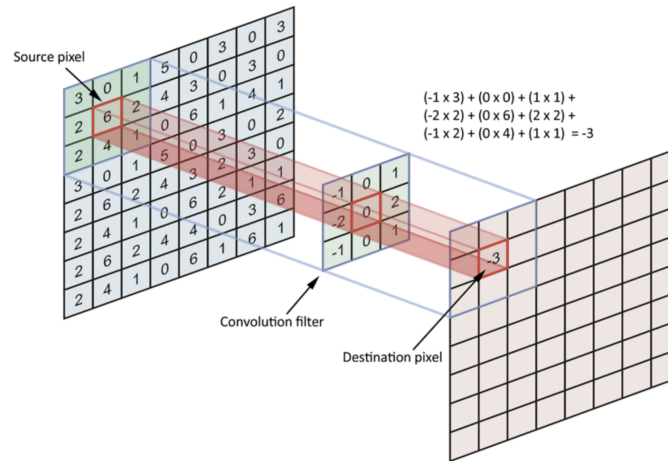


Figure 4: Visualisation of CNN convolution layer, from (Stewart, 2019).

Wav2vec takes raw audio files as an input, processing data through two CNNs. The first network performs feature encoding, and the second performs feature aggregation. The model then outputs a general representation, a matrix composed of 512 layers of features representing the audio input along 10ms time steps. This differs significantly from the feature extraction system used in DeepSqueak or MUPET, for example. Most prominently, it outputs far more features, and these features are not handpicked or manually defined. This serves the purpose of escaping a potential area of human bias in previous analysis tools as previously discussed.

Performing extraction and then clustering would be done with human speech data to compare to a known and expected value for evaluation before use on USVs. With Wav2Vec's large general representation output, this step would also be essential to identify the features of relevance at the level of abstraction useful for syllable categorization, before configuring the system for use with USVs. Different layers of a CNN are effective at extracting features at different levels of abstraction, exemplified in figure five below, and understanding which are the best fit for our form of data would be very important in achieving success.

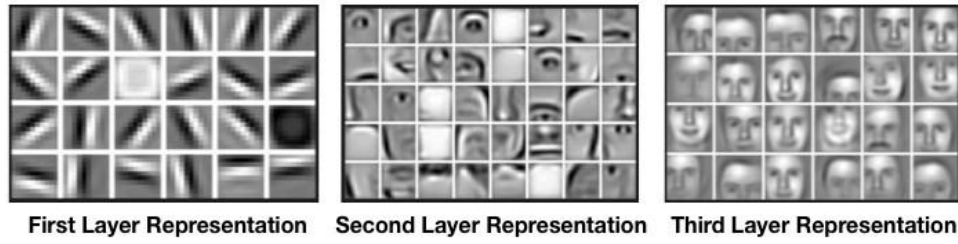


Figure 5: Visualisation of CNN feature layer representations for facial recognition, modified from (Lee et al., 2009).

To cluster data in this solution, the k-means algorithm as used in previous tools was selected. It represented a fairly simple implementation challenge, with a large amount of prior use backing it, which was considered enough to justify continuing with it over including a new algorithm; although we were conscious of criticisms laid out in (Goffinet et al., 2021). We would also go forward with using the Elbow Method to identify the best fit of K , without overfitting the data. The proven track record of use in functional systems in the same area of animal call categorization of this technique, along with its broad acceptance in the field and the complications presented by implementing a new solution, were the justification for going forward with an elbow method-backed k-means clustering.

The k-means clustering algorithm groups data points into clusters based on their euclidean distance to a certain number of centroid points, with arbitrary dimensionality depending on the amount of features of the data points. The number of centroids, “ k ”, is given by the user, and the algorithm finds *a* best fit of the points into that number of clusters. This is not necessarily *the* best fit, as the initialization of the algorithm can produce different results, finding and returning different local maxima of best-fit.

The algorithm starts by randomly positioning a set of centroid points and assigning all data points to the closest centroid. Then, the centroids are moved to the center of all the points assigned to that cluster. This process repeats, assigning all points to the closest centroid, and then moving the centroid to the center of those points, until the results converge and the centroids no longer move (or, if a maximum number of iterations is reached). The results following this convergence are a maximum of best fit, and some implementations will perform this process multiple times with different random seeds in order to potentially improve results by finding better local maxima of fits, generally as measured by a fitting’s “inertia” or total sum of squared error—a value calculated from the distances between points and their centroids (Chong, 2021).

While the K Means algorithm groups the nodes into a best fit for a given K amount of clusters, it is the elbow method step that actually identifies the best value of K when that value is not known or is undesirable to define. A range of values to try is defined, and for each value a K Means clustering is performed. The total sum of squared error, or inertia, for that value of K is recorded, and once all values in the range have been tried with the clustering algorithm, a graph can be created of the reduction in error as K increases.

The amount of error will naturally decrease as more clusters are introduced in most cases, as the clusters will each be more tightly fitted. When there is one cluster for every node, the theoretical total error is zero. However, as the amount of clusters increases the risk of overfitting to the data grows. As previously described, the elbow point that maximizes goodness of fit while minimizing risk of overfitting can be found mathematically, by selecting the point along the graph curve where straight lines best fit the curve to the left and the right. The elbow can also be identified visually, by inspecting a graph manually, giving the user an idea of the right range of values, perhaps for refinement in another form of optimization such as finding the k value with the best score in another fit metric.

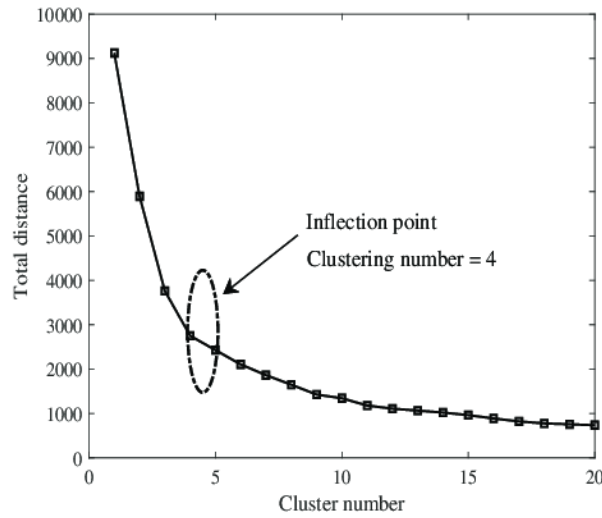


Figure 6: An example of a graph of k clusters against total distance/error. The elbow identifies the best fit, from (Zhang et al., 2020).

Because a large number of iterations would be necessary in the testing stage, and in order to continue to automate each step as fully as possible, visual inspection would not be used to identify the best k value. In this design, the kneedle library for python was selected to automatically perform the elbow method identification of the best fit K. This means the full process of finding a best fitting set of clusters for the given data, including the decision of how many clusters there would be, are out of the hands of the user, as desired.

In order to identify the relevant layers of features for our goals, that is those that are most informative regarding approximately phoneme or syllable-level data structures, a large amount of American English speech data would be processed through Wav2Vec, and the features across all data points for each layer would be processed through k-means clustering using the elbow method, as individual sets of single-value nodes, using a range from 2 to 100 for K values to test. This would output a best-fit K value for each layer, intended to better inform the next steps by showing which layers more closely correlate to expected values of K. The implementation of this process would later turn out to be seriously flawed, as discussed below.

American English speech was chosen for this step because it has well-researched sub-unit categorisation in existing literature. ARPAbet is a comprehensive scheme of 48 phonemes used in American English, with a small amount of regional variation (Mines et al., 1978). The librispeech dataset of speech was chosen for its large size and relative cleanliness, as well as its ease of access. Layers with a K value in the range of around 45-55 (to account for regional variation) would theoretically indicate that their features are valuable for the relevant scale of categorization tasks.

The initial results of this step indicated problems in the design and implementation of the solution. K Values across a number of layers were very similar, with a range of only a few values, all in the range of 11 to 13. Because processing times for this step are very long, to explore the problem a sample across all 512 layers was taken, calculating the best fit k at each 50th layer. Data was stored to produce each graph manually, and automated results continued to show a value of between 11 and 13 as the best fit for all layers.

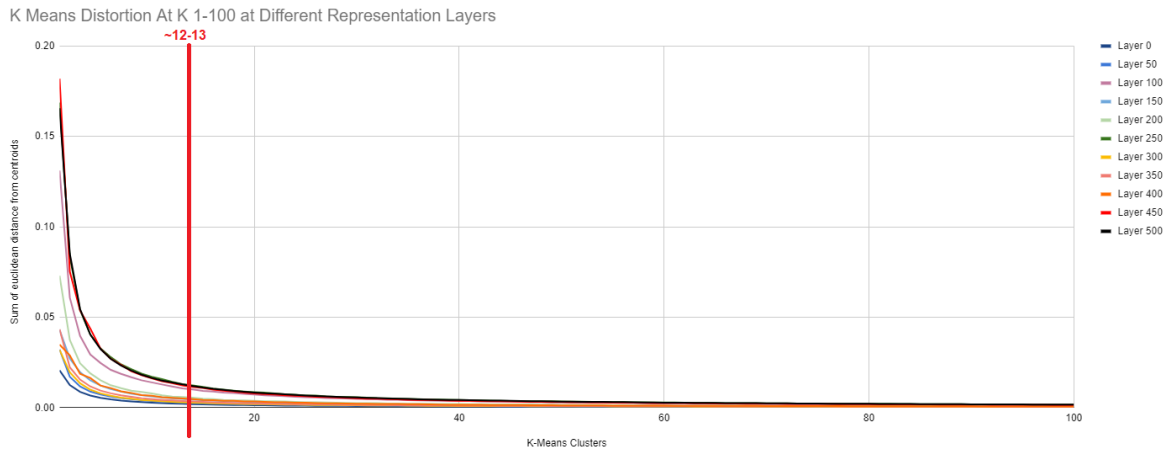


Figure 7: Graph of the elbow method applied to each 50th layer of a wav2vec representational output of speech individually.

As can be seen, while there is a clear difference in the level of error across the layers, particularly at small values of K , the elbow point does not differ significantly. Beyond the first twenty or so values of K , the difference in error is so small between each value as to be unnoticeable at this scale. To confirm the problems that this solution was facing, a second run of this test was performed for only K values in the range of 25-75. While this naturally moved the mathematical elbow point into the low 40s, again the best fit K for all layers was within the same range of a handful of values.

The lack of differentiation between clustering results at different layers of abstraction in the general representation introduced a number of questions as to the effectiveness of the solution design. The clustering was unable to highlight a layer of relevance to the identification and categorization of syllable-level units in the extracted general representation of human speech, and producing values from processing USV calls the same way could not be done with any confidence in the accuracy or informativeness of the results.

Later evaluation of this initial technique, following significant redesigns and better results, would indicate several key errors in the understanding of the underlying processes at this stage. In fact, while the design intended to cluster the results of wav2vec's CNN *layers* and find the most appropriate to extract results from, the implementation was performing this cluster and elbow method process on individual CNN *features*. This is analogous to attempting to identify which pixel of a 512 pixel image best describes what that image depicts, obviously a futile task.

This revealed critical misunderstandings of the structure of a wav2vec general representation output format. More research and experimentation with the system was clearly necessary. Furthermore, while it was considered that the analogy of USVs to human syllables or phonemes is useful at a basic level, it was also decided during this reconsideration stage that a system's performance on human speech could only marginally indicate towards its performance on USVs in any case, and an improved design should be evaluated on its performance using USVs first and foremost. This would also save time in a redesign in order to stay within the bounds of the project timeline.

Designing and implementing many parts of this initial solution was not time wasted. Following this experimentation, a much better understanding of the inner workings of the wav2vec system, as well as of clustering algorithms and the past research into USVs, was achieved. Many lessons could go on to be applied in a revised design. This initial design ultimately served well as a first test of many partial components in a theoretical successful solution, representing an experimentation stage that would ultimately lead to refined results and a better final software artifact.

However, this initial solution was far from the mark in performance. While individual parts of the implementation, including feature extraction and the clustering algorithm itself functioned as intended, key errors in the transition between these stages compromised the overall design. Rather than restructure this solution and patch up these errors to attempt to find acceptable results with this relatively simplistic design, this presented an opportune moment to move from basic experiments into designing a better solution from the ground up.

Improvements were possible in every component, now that the foundational understanding was better laid to move towards them. Later, higher performing models of wav2vec could be used for feature extraction now that the basic structure of the model was better understood, and new clustering methods able to handle more complex data without the need for a defined number of clusters could be considered as an advancement on past tools.

New techniques for USV selection could also be implemented based on phonemic segmentation processes used in English speech, and clustering of these selections would now be able to occur over a wide range of features, having addressed the single-feature clustering error of the initial design. The revised design that emerged as a consequence of these initial experiments and their lackluster results would go on to be implemented fully, and ultimately proved much more effective.

4. Revised Design

Feature Extraction

Moving out of the wav2vec ecosystem at this point was considered, to explore other possibilities. Most seriously considered, the method of unsupervised lexicon discovery put forward in (Lee et al., 2015) describes a different technique for achieving similar goals. Timesteps of audio data are converted to 13-dimensional Mel-Frequency Cepstral Coefficients, and a model is trained to use these to predict the boundaries between segments. These segments can then be processed through a noisy-channel model to account for variability of pronunciation, producing standardized representations which can be inferred for each utterance using probability-based adaptor grammars.

This unsupervised lexicon discovery technique was considered interesting, but would require dramatic restructuring, and effectively a reset from the ground-up in terms of building a full understanding of system internals for implementation. Additionally, moving to such a system from the fundamental processing pipeline of USV detection, then feature extraction, and finally call clustering would represent a significant shift from the expectations of potential end users, when the effectiveness of USV clustering remains an open question that researchers are likely to want to further explore.

Finally, while this technique achieved reasonably impressive results in F1 scores for phoneme segmentation on select data from a few lectures, wav2vec-u showed comparable or better results as a phoneme error rate against the larger TIMIT dataset. All up, there was no significant performance improvement to expect for the tradeoffs in user expectation and implementation difficulty, so retaining a wav2vec-based system was determined to be the most effective option going forward.

While the initial solution ultimately only tested on human speech data as a proof-of-concept before it was decided to move on to a significantly revised design, the finalized solution would be working with USV data from the very beginning. Focusing on evaluating effectiveness by actual results in the specific data domain, rather than testing on human speech to build some kind of general but possibly misapplied confidence, would likely lead to stronger actual confidence in the real final results. One simple test to apply to both human speech and USV data to evaluate the effectiveness of the new segmenting step was conceived, and is discussed in detail below.

Apart from this test of call/phoneme segmenting to check basic indications of performance, USV calls exclusively would be used in processing and system testing. The dataset used was provided by the Behavioural Neurogenetics Lab at Victoria University of Wellington. Several hours of recorded play sessions between pairs of male rats and pairs of female rats would ultimately provide the main source of USV audio in testing. Unlike with human speech audio, significant preprocessing would be necessary in order to use a wav2vec model with this data.

To fully understand the necessary preprocessing, a specific wav2vec model needed to be selected at this stage for feature extraction. Ultimately, wav2vec2.0 was chosen for its improved performance against the original model, and for its use as a feature extraction tool in the later wav2vec-u system, which included an unsupervised phonemic segmentation process discussed in (Baevski et al., 2021). This positioned wav2vec2.0 well as the natural improvement on wav2vec as used in the original design, which while simple to use was not as performant and could present complications in adapting later feature processing steps to.

Theoretically, it would be possible to train a wav2vec2.0 model from the ground up on USV data, to get the best possible feature extraction performance. However, these models train on much larger amounts of data than available in this project to achieve such levels of performance, so it was

determined that a practical solution would be using a generalist version of the model called XLSR-53, which was trained in bulk on fifty-six thousand hours of audio in fifty three human languages.

Being so broadly trained made XLSR-53 the best option, as it is intended for effective use on a broad range of lexicons. A model specifically trained for English or another human language could prove less effective on a different lexicon than a model that is slightly less effective on average but performant over a much wider variation in input data. This makes it the best option for use in USV analysis, which is of course not closely comparable to any specific human language. These generalist models can also be theoretically fine-tuned to improve their effectiveness for a specific form of data, but fine-tuning requires data labelling, that is to say it requires pre-defined true values, making this an impossibility for this project.

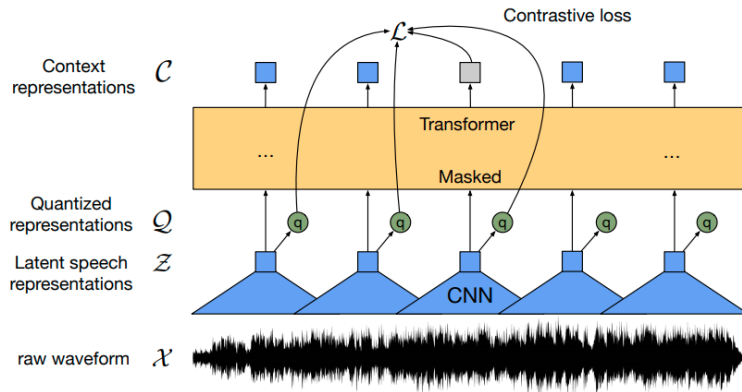


Figure 8: Internal structure of a wav2vec2.0 model, from (Baevski et al., 2020).

Wav2vec2.0 models are similar to those from the original wav2vec, making use of seven blocks of convolutional neural networks that take an audio signal as an initial input. These CNN blocks work to extract a set of encoded features comprising speech representations for 20ms timesteps (with a receptive field of 25ms of data), and positionally embed these representations relatively (Baevski et al., 2020). The primary innovation present in wav2vec2.0 that is new from its predecessor is the transformer model that follows this CNN step.

Transformers were first introduced in (Vaswani et al., 2017), and continue to represent part of an ongoing paradigm shift in AI techniques as described in the recent comprehensive report of the field, (Bommanasi et al., 2021). Transformers are a sequence-to-sequence model, taking a full sequence of data and processing it simultaneously to output a new sequence of data, much faster than iterating over a sequence as in older systems. They use a mechanism called *attention*, which describes the relevance of certain data to other data according to a particular definition, though by using multiple attention heads, multiple definitions of relevance can be considered.

Wav2vec2.0 models use a transformer to take the sequence of audio representation timesteps and fully contextualize their data across the entire sequence. With the generalist XLSR-53, there are twenty-four transformer blocks, and sixteen attention heads. This series of transformer blocks outputs a sequence of timestep representations that have been fully contextualized, which is excellent for our case, where we would like the results to be able to account for the possibility of inter-call context being significant, if the results support such a finding.

(Baevski et al., 2021) has also already gone through a process similar to a correctly-implemented version of that attempted in the initial solution, to identify the best blocks of the transformer representation to take data from for a following phoneme-level segmentation step. They found that the sequence output of block fifteen is most effective, with the lowest phoneme error rate on testing with

the Librispeech and MLS datasets in eight languages. Consequently, we will go forward with block fifteen sequence outputs in this project.

Data Preprocessing

As mentioned, to use this feature extraction model, we first need to apply a series of preprocessing steps on our USV data to make it acceptable. The first issue to address is the disparity in sampling rate. XLSR-53 and other wav2vec2.0 models are trained to function on audio data with a 16-kHz sampling rate, but to record our USV data extremely sensitive ultrasonic microphones are used. Our USV data is recorded at 192-kHz. The model will not accept data at this sampling rate, so adaptation is necessary.

Simply downsampling our USV data by a factor of twelve could prove catastrophic in terms of the loss of information, so this option was ruled out very quickly. Instead, the sampling rates of the audio file headers could be altered using a script, without altering the data itself in the files. This would in effect make these files twelve times longer and lower pitched, but it does allow the model to take in all the information. Later steps can account for the temporal difference, and making these short ultrasonic USV calls longer and lower in pitch (in fact, this adjustment does make them audible to humans) actually makes them more alike the human phonemes these models are trained using.

With the sampling rate addressed, the other major preprocessing consideration is noise, and the general isolation of the relevant signal. Again, USVs are recorded using very sensitive equipment, meaning a lot of noise gets picked up along with actual calls. Beyond background static and the hum of the microphone equipment itself, even minor bumps or the sound of a researcher breathing can register as major spikes in sound amplitude. Isolating USV data from noise was shown to be a key challenge to tackle in prior works, and there are a number of potential solutions to consider.

DeepSqueak's style of training a model to pick out sonographic indications of USVs is effective, but it requires user input to weed out noise. A solution more similar to that of the predecessor MUPET may actually be preferable when a primary goal is total automation. With this in mind, a two-step noise removal process was designed. First, a band-pass filter attenuates out signals above and below the relevant frequency band for affective USV calls. Then, a spectral gating algorithm is used to eliminate background noises.

This set of noise reduction steps should effectively isolate USV signal from background noise where it exists. It is not perfect, as particularly loud bumps and other such sudden noise spikes may reach above the noise threshold of the spectral gating algorithm. Designing a solution to refine this further in an automatic way is difficult, as exemplified by the manual noise refinement processes of earlier solutions. Ideally, users will have access to data clean enough that these two noise reduction steps will be effective enough alone, but at the same time, ideally this design should work for data that is *not* so clean as to require no further attention.

This is a problem that was considered for some time. Possible answers included training a new machine learning model like that used in DeepSqueak for sorting USVs from other sounds, though it was unlikely enough data was available in this project to train an effective general model, and this would require the same sort of designer-determined labeled training data for a USV/Noise classifier that we sought to avoid. An alternative possibility was to design an algorithm that excluded any signal with high-amplitude sound across a large range of frequencies from the data, but this would only capture a subset of noises with that characteristic while presenting a number of implementation complications and difficulties.

Ultimately, designing a system to automatically pick out surviving noisy bumps from the rest of the otherwise-isolated USV signals was determined to be outside the realm of achievability considering the scope of the full project in need of design and implementation. At this stage in the design we

prioritize an imperfect but functional module, which will perform fairly well on clean data. Noise reduction is kept strictly detached from other processes and called as a separate script, in order to make any future improvements in this area as simple as possible to implement. It is left to the user’s discretion whether they would like to manually clean their data further after the automatic noise suppression, by editing it to cut out surviving noise they can visually pick out, but this is not expected.

Following noise reduction, there is one final preprocessing step before the data is acceptable for a wav2vec2.0 model. This is silence removal, to get rid of the large gaps between calls and exclusively provide the model with actual USV call signals. This is called for in technical documentation of the model, which also provides a recommended off-the-shelf solution that the designers of wav2vec2.0 have used in their own processes. This solution, rVAD, will be used for silence removal in this project. It represents a smaller challenge to implement than a new solution and comes recommended by the makers of the feature extraction model, two strong arguments in its favor that led to its adoption.

Representation Segmentation and Feature Processing

Following these preprocessing steps, the USV audio signal is finally ready to be accepted by our XLSR-53 model which is described above. Through the CNN feature encoding and the transformer sequence contextualization, we get as an output fully contextualized, 1024-feature representations of the signal at 20ms time steps. Due to our sampling rate adjustment, these represent 1.67ms of real time. By inserting a mean-pooling process across each six adjacent steps at this stage, we get representations of each 10ms of real time.

This means steps represent half the time they do in similar processes for human speech, which is actually ideal—affactive USV calls have about half the average length of a human phoneme, so this balances out to considering units of data at about the same speed. At the end of feature processing we can also increase the mean-pooling done for human speech by a factor of two, which restores effective real-time to our final results.

The feature processing to take our contextualized timestep representations and conglomerate them into segments that each hold and represent one USV call is based on the processes for phoneme segmentation in wav2vec-u, as outlined in (Baevski et al., 2021). Wav2vec-u ultimately uses text data at a later stage to train a generator model for actually classifying these segments, which is unavailable for USVs. However, we intend to use a clustering algorithm for unsupervised classification, so we can happily use similar processes to generate our actual segments, and then diverge from wav2vec processes at that stage for classification.

Segmentation based on wav2vec-u processes is advantageous, as it is well defined and provides open-source examples of implementation. Additionally, it was specifically designed with the general representations given as an output by a wav2vec2.0 model in mind. Due to both the advantages presented for ease of implementation and its proven effectiveness with our chosen feature extraction method, consideration of swapping to another segmentation solution is largely unnecessary. However, the solution should still be designed in such a way that this segmenting solution is detachable from the feature extraction step, for ease of maintainability and improvement in the future if more performant solutions are later found.

The timestep segmentation process includes a number of substeps. Some can be largely brought over as implemented in wav2vec-u, but many require at least structural changes to allow for the initial mean-pooling, and there are several opportunities for GPU memory optimization that will be necessary for running this software on systems available to low budget academics, likely a key demographic of potential users. Generally, timesteps can be clustered using k-means, and when the cluster of the next timestep is different, segment boundaries can be introduced. All timesteps of each

segment established this way can be mean-pooled together, followed by a final adjacency-based mean-pooling to account for unsupervised noisiness in boundary-setting.

Principal Component Analysis is also used during this process, to orthogonally project the data into fewer dimensions that maximize the retained variance. This serves to simplify the data for later computational purposes, as well as helping to escape the *curse of dimensionality*. In essence, more dimensions leads to more evenly distant data across those dimensions, making classification tasks more difficult overall. PCA reduces the number of dimensions to a predefined and more manageable amount, mathematically maintaining as much variance information from true features as possible.

For wav2vec-u's human phoneme segmentation, initial timestep clustering uses 128 clusters, and PCA reduces to 512 dimensions. For the more structurally simple set of USV calls, it is expected these numbers may be lower. Rather than manually defining these values initially, they will be optimized at the implementation stage to find the best result based on a simple metric of average segment length.

Basic Length Test

Following the process outlined above, raw audio signal will have been converted into a set of segment representations of many features. There is no simple test to evaluate how well these segments are actually selecting individual USV calls, and manually analyzing a large enough amount to produce a confident measure of correctness could prove impractical. One indicator that can be measured without a detailed internal analysis however could offer an indication as to whether this segmenting is on the right track or not. That is the average length of each segment, and how that compares with expectations of average USV call length.

This is the one place human speech data is used in this design, for calibration and comparison with USV data. This model can be run against human speech data, and the resulting total number of segments is compared to the runtime of the input to find the segments per second. Assuming each segment represents one phoneme, the average length of a segment should roughly equate to the average length of a phoneme. This test can then be applied to USV data, with the expectation that segments should then equate to around the average length of an affective USV call. This value matching our expectations would not necessarily be confirmation that call isolation and selection is flawless, but were this value far from expectations that would be a strong indication of the reverse, that the segments are poorly representing individual USV calls.

The results of this test are discussed in the implementation section. In summary, findings were almost exactly on target for phonemes, and well within the healthy boundary for USV calls without much parameter adjustment necessary. This confirmed at least that the average segment correlated well with the average call in terms of length, and more precise evaluation by manual examination of call selection compared to generated spectrograms can be performed during final evaluation to assess the effectiveness of the system's call isolation more directly.

Segment Clustering

If we can achieve a reasonable level of confidence in the feature extraction and processing stages, the last key piece of the puzzle is the unsupervised classification of our resulting segments, which represent individual USV calls. This will be performed by a clustering algorithm, the most common form of unsupervised classification without true data labels to train on, and the expected solution for users to study in this field. The open question at this design stage is not clustering or another classification solution, but rather whether to continue with an implementation of k-means clustering as in previous tools and the initial solution design, or whether to adopt an alternative clustering algorithm seeking better performance.

(Goffinet et al., 2021) shows that USVs are unlikely to cluster into tidy geometric blobs, and while clustering them shows better fit than clustering random noise using MUPET’s implementation of the k-means algorithm, it is not significantly better as there is a broad spread of data across the space at varied densities. The UMAP projections provided (figure 3 in this report) can not conclusively tell us whether USVs form separated shapes in higher-dimensional space or not, but if they do these shapes are clearly more complex than those of other examples such as bird songs, and may even spatially overlap with each other.

The algorithm HDBSCAN, which (McInnes et al., 2017) provides a number of useful technical documents and implementation examples for, was selected as a potential alternative to the industry standard k-means clustering approach. HDBSCAN is an extension of DBSCAN, and functions by calculating a mutual reachability distance between points that considers data density, then constructing a minimum spanning tree of all data points. This minimum spanning tree can be processed in reverse to build a hierarchy of clustering, which can then be condensed based on given hyperparameters, and finally from this can be selected the set of clusters with the highest *stability*, a measure of the strength of cluster membership for each point in the cluster.

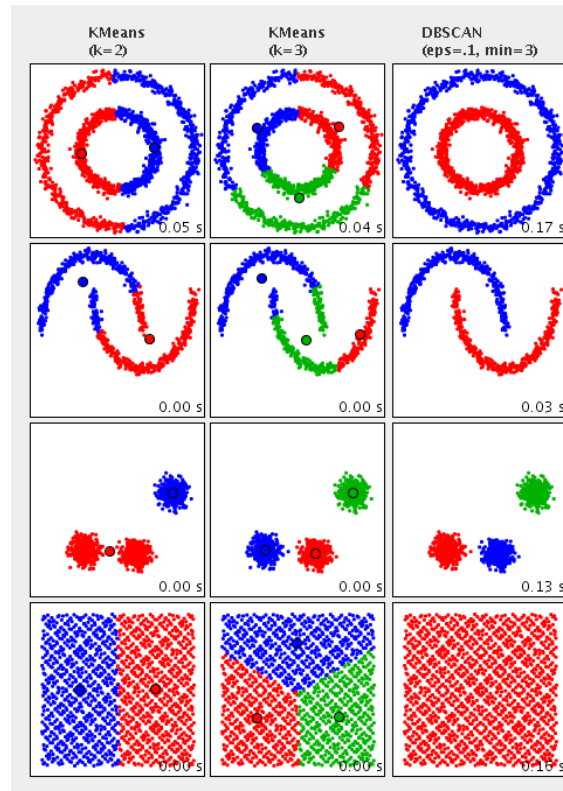


Figure 9: Comparison of k-means and DBSCAN (which HDBSCAN extends) clustering results, from (Clustering, 2022).

HDBSCAN has several major advantages over k-means. It can identify clusters of different densities in the same data, as well as clustering complex shapes that might spatially overlap. It is also intrinsically a “fuzzy” clustering algorithm, meaning it can classify certain data points as noise, which do not fall into any cluster. This can be useful if genuine noise segments do slip through into the clustering stage, as they will generally be feature outliers, and this is also helpful for finding useful clusterings of higher density within a spread of data that is not clearly separated in euclidean distance.

HDBSCAN does require predefined hyperparameters to function, but these can be automatically optimized for the best relative “validity score”, a version of Density-Based Clustering Validation (Moulavi et al., 2014). This is a metric of the goodness of fit of a clustering that is calculated based on

the density of clusters and the decrease in density between them, rather than a basic euclidean distance. This allows for measuring the goodness of fit of arbitrary shapes of clusters, essential for an algorithm like HDBSCAN to be measured in an informative way. For the sake of generating experimental results to use to evaluate this project, hyperparameters can be automatically optimized for the best relative validity score in a method analogous to the elbow method. End users would be able to use automatically generated hyperparameters, but they would also be able to define these directly, providing the user valves to turn at their discretion if they would like to generate other clustering results to look for one that best fits their data and purposes.

While this algorithm does take these predefined hyperparameters to function, it is notable that it does not require a predefined number of clusters to sort the data into. Unlike with k-means, this value is automatically determined during the clustering process, a better fit for the goal of less human decisions in the process (the hyperparameters mean this is not perfect automation, but with this algorithm humans could only explore the performance of different abstracted hyperparameters rather than having a direct hand in defining the number of clusters to sort calls into).

K-means has the advantage of being notably computationally lighter, as well as being somewhat more simple to use to classify new data later on, by simply finding the nearest cluster centroid point to the new data point. However, it is still possible to achieve later symbolic representation transcripts using HDBSCAN, by saving the clustering object and using soft clustering predictions to calculate the probability a new data point falls into each given cluster. In sum, the extra computational cost of HDBSCAN was considered worthwhile for the added robustness in dealing with differing densities and complexities of cluster shape, and this is the clustering algorithm in the final implementation.

Symbolic Representation Transcript Generation

As mentioned, symbolic representation for evaluation using HDBSCAN clustering is possible. A slightly modified version of the existing processing scripts for USV audio data can take new arbitrary USV recordings at a future time and process them in the same way the bulk data used to generate a categorisation scheme was processed. The resulting new segments, each of which represents a new USV, can be given as an input for soft clustering prediction. The cluster assignment with the highest probability can be recorded as the likely call category, while the full probability information can also be recorded for any deeper statistical analysis a future user might like to perform.

This technique can be used to allow a single command-line argument to take USV audio data and process it fully into a transcript of call codes. In order to account for differences in sourced data for different users (for example, USVs recorded at different sampling rates) and to allow for easy changes in the noise and silence removal steps, preprocessing is not included in this command. A set of commands is given to perform the provided preprocessing steps, but these can be very simply swapped out, and any USV audio that has had its sampling rate adjusted to 16-kHz and received noise and silence removal of some kind will be acceptable to the transcription system. Additionally, all tools are provided for the user to create their own categorisation schemes in the same way the experimental one was created, automating this process for researchers who might have their own datasets or seek to tweak certain parts of the process.

Once the transcripts are generated, it becomes possible to qualitatively evaluate the performance of the system by comparing these to a spectrographic visualization of the same signal. This will provide a better idea of how well the system selects and isolates USV calls into segments, as well as permitting an evaluation of how well the experimentally generated categorisation scheme (and future schemes generated by users) classifies different USV calls in a way useful to research. With these steps all complete, a fully functional system will be able to take unlabeled USV data, and without any human intervention necessary generate a new scheme of call types, as well as providing the tool to apply that call scheme to arbitrary new data to transcribe, for evaluation and practical research use.

5. Final Implementation

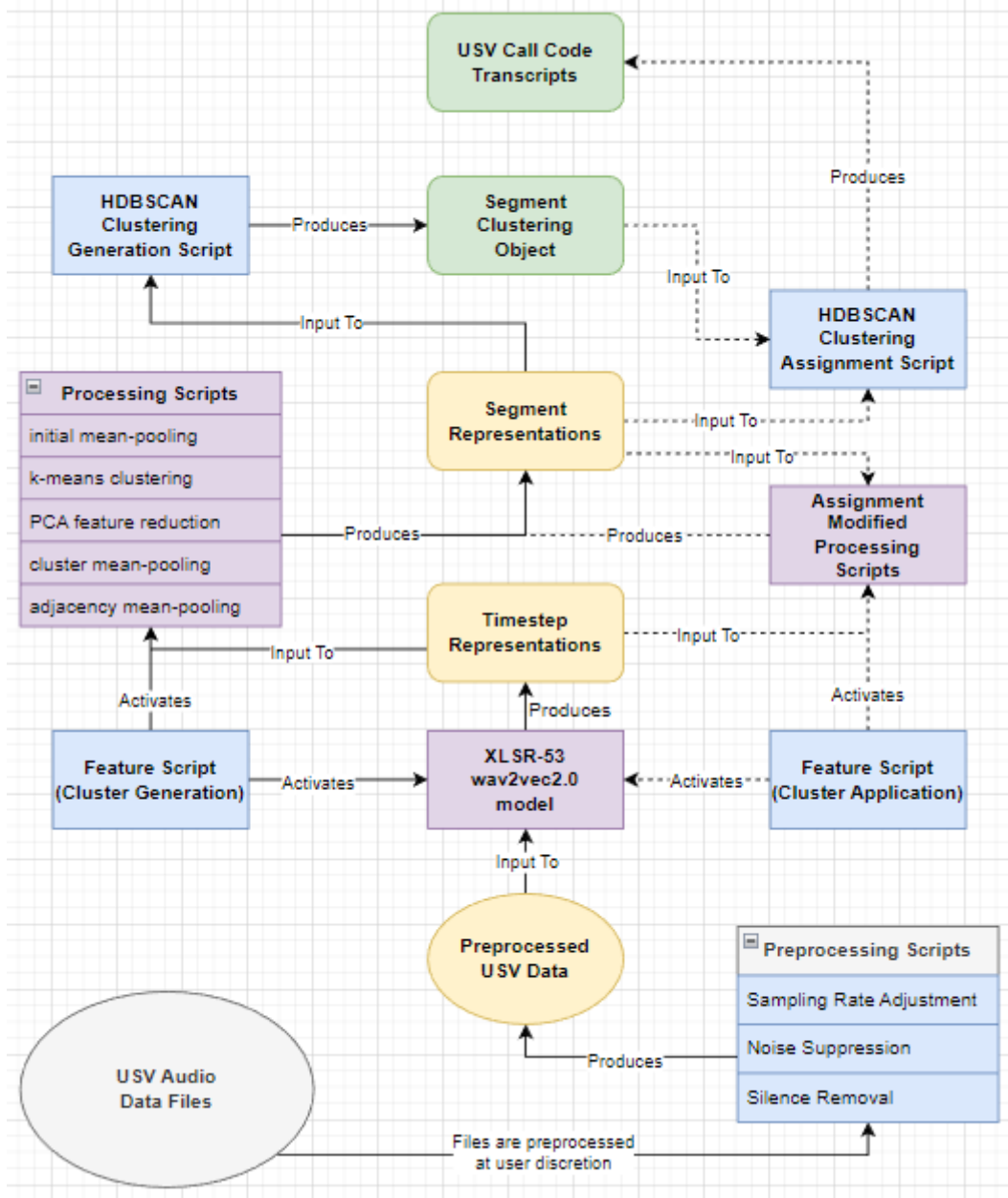


Figure 10: A diagram depicting the structure of the final software artifact.

Depicted in figure 10 is a diagram of the implementation structure. Marked in blue are scripts run by a command of the user, purple are scripts, algorithms or models that are separate but are run internally, called from another user-activated script, in yellow are internal data stages, and in green are potential final outputs. Black arrows indicate processes for the categorisation scheme generation use case or both use cases, while dotted arrows indicate processes for the use case of applying an existing categorisation scheme to new data for generating a symbolic representation transcript. All scripts are run in a project-specific python virtual environment that comes with the software artifact, with the correct installations of relevant libraries and modules.

Preprocessing occurs in three separate scripts which are optionally activated by the user via command. This is because not all USV data is alike, and different users may have significantly different

preprocessing needs, so it is advantageous to have this system as modular and customizable as possible. Some users may have USV data that has already been preprocessed in some way, or requires additional steps to be ready for feature extraction and analysis. For the general case however, scripts are provided to automatically handle key preprocessing steps and clear documentation in an attached `readme.md` file helps to guide users in using the preprocessing scripts they might require.

Before even the preprocessing scripts, another short shell script is provided that uses the `sox` library to convert audio files from `.aif` to `.wav` format, used in `wav2vec`, in bulk. This is purely a convenience for users, as many existing solutions are perfectly acceptable for this task, and while `.aif` is a common format for raw USV files, some users may have raw audio in other formats. The data this project had available was `.aif`, so this convenience script converts from that to `.wav`.

Having attained files of the right filetype, a user is likely to need to adjust the sampling rate. Unless other preprocessing has already taken place, USV audio is typically recorded at a much higher sampling rate than 16-kHz. A script is provided to edit file headers in bulk to this sampling rate without modifying actual data, if this step is necessary for the user's files.

Following this, the next script that would be run in the typical use case is for noise suppression. Again, it is very possible users will seek to use data that has already been cleaned of noise, in which case running this script would prove unnecessary while losing a marginal amount of valuable USV information. Other users might seek to use a different noise suppression system to the one provided in this system's full solution, due to personal preference or later identification of more effective noise removal methods. So this script is optional, though strongly recommended for files that are not otherwise cleaned of noise.

The noise removal script runs two methods on the data. First, an implementation of a FIR band-pass filter, beginning attenuation at above 68-kHz and below 34-kHz, to isolate the relevant frequency band. These values proved effective through experimentation, but they are single variables to replace if changes are desired. Following this, (Sainburg 2019)'s noise reduction algorithm is applied to the data. This spectral gating-based noise suppression functions similarly to the noise removal of MUPET, working by internally creating a sonogram of the audio and calculating noise thresholds for each frequency band of the signal. These thresholds are made into masks that gate noise below the threshold, varied by frequency. This has proven effective at removing background hum and static almost entirely with the right parameters.

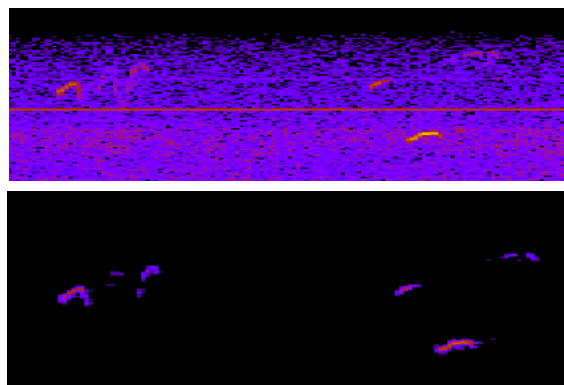


Figure 11: Example of the effect of the noise suppression script on a USV file, visualized spectrographically.

Static, steady hum, and sounds outside the relevant frequency range are eliminated. There is a slight degradation in USV quality as a tradeoff, but this is a very worthwhile exchange, especially given the necessity of silence removal (the performance of which is significantly improved with less noise and hum) to condense the audio to USV calls only, for the current model used in feature extraction.

Otherwise a significant amount of segment representations would encompass empty noise. This noise reduction solution is effective on steady background static, but occasional loud bumps will not be filtered out. Clean data collection will naturally prevent this posing a serious issue, but on dirtier data users can also edit their files directly after this stage to cut out remaining bump noise. For several USV recording clips used in experiments in this project, this was done in order to improve results.

Following noise suppression, the final preprocessing step is silence removal. As recommended by the wav2vec-u github, rVAD is used as an off-the-shelf solution for this process. Silence removal is necessary for the best function in the wav2vec model, to ensure timesteps later processed are not representing empty air. This script like the others is detached and optional to account for differing data or improved processes, but the large majority of users will likely need to run it to prepare their USV audio for wav2vec processing whether or not they have used the provided scripts for prior preprocessing tasks.

Using rVAD proved implementationally simple after a difficult dependency installation process (which is in large part why a distinct python virtual environment is provided, so the user can avoid any of these complications). However, this would ultimately be one of the slowest parts of the entire system. In fact, over a third of the total time it takes to process data from raw audio to final clustered segments is spent in rVAD silence removal. This represents one of the largest areas of potential improvement, going forwards, but the method is ultimately effective.

Following whatever preprocessing is necessary for an individual user, they will have USV data ready to use in the system proper. There are two main use-cases, each of which comprises two user-activated scripts. The first use case is to generate a new clustering/categorisation scheme. To do this, the user first activates a script for the feature extraction and processing pipeline. This script itself comprises a list of other scripts to activate in succession, first activating the XLSR-53 model to extract features from the preprocessed audio, and then running through a series of feature processing steps as outlined in the design section to turn these representations into segments.

Significant parts of this script and its subsidiary scripts could be adapted from the design of the segmentation process in wav2vec-u. However, a number of optimizations were necessary to ensure the system functioned on a single modest GPU, to make sure access to the system would not be limited to high-budget research groups only. For example, there are several points in the original scripts where features are regenerated from the audio data and written to the GPU as new tensors with CUDA for a subsequent processing step. Instead, in this implementation the features are written to disk as a numpy array file the first time they are generated by XLSR-53, and read again as necessary as numpy arrays without regenerating. This is slightly slower and more expensive in terms of disk space, but the tradeoff for functionality on more accessible GPUs is significant.

The other most significant change from wav2vec-u's implementation was the addition of a new first timestep mean-pooling step (where each six steps are pooled), and modifications to the final segment adjacency mean-pooling step (where four, rather than two segments are pooled), that addresses the temporal difference caused by the twelve-factor change in sampling rate. This means data is half as fast as real time during processing, which is appropriate as USV calls take half as long on average as a phoneme, and final results are restored to real time. Several other scripts also needed to be modified to function with these changes to the mean-pooling.

At this point the segmentation test discussed in the design section could be implemented to in part verify the effectiveness of the system. The accepted value of around 14 phonemes per second on average in English speech, with a normal range between 10-15, has long-standing backing as far back as (Lee, 1950). Knowing to expect an average affective USV call length of around 30ms (Thompson et al., 2006; Seidisarouei et al., 2021), we can calculate the expected calls per second in one second of condensed USV signal. This gives us the values to seek in a comparative test.

	Trial Length (S)	Segments Identified	Segments per Second	Expected Segments/S
<i>English Speech</i>	347.18s	4,954	14.27	10-15
<i>Affective USVs</i>	53.3s	1,811	33.98	25-35

Using a randomly selected subset of data from the Librispeech dataset for American English human speech, and a randomly selected subset from the recordings provided by the Behavioural Neurogenetics Lab for USV calls, both were sequentially and separately processed through the data preprocessing, feature extraction, and processing and segmentation steps as implemented. The runtime of the data in each test and the total number of segments identified is provided in the table above.

As can be seen, the average length of a segment is comfortably within the bounds of our expectations both in the case of human speech segmenting into phonemes, and USV recordings segmenting into individual affective calls. This does not necessarily confirm that segmentation is functioning correctly, but getting roughly the expected number of segments as a result is a positive indication with regards to the effectiveness of our mean-pooling measures to account for temporal differences. How effectively individual calls are being separated into individual segments will require manual analysis, more easily performed after transcripts are generated.

Finding the segmentation to be acceptable, these segments now need to be clustered. This is where the second script is activated by the user. A python library that implements HDBSCAN clustering (simply called hdbscan) is used, as it is a fairly efficient implementation of the algorithm. The segments, which in the case of USVs each have 32 features after PCA (a value optimized for best performance in experiments, but simple to change if desired), are turned into data points for the clustering algorithm. Hyperparameters, primarily the minimum cluster size, are also optimized for the best relative validity score, though again these are simple to adjust if the user wants to find different results.

The clustering library returns a cluster assignment for each segment, or assigns a segment as noise which falls into no cluster. Taking the highest value cluster code assigned tells us how many categories the algorithm identified, and this value is presented as an output to the user immediately. The clusterer object itself is capable of probabilistic predictions of assignment for new data following its processing of the main dataset, and this object is the main product of the clustering script. It is saved as a file using python pickling, and can be loaded by another script to use this clustering result on new data.

For the second use case, the application of a generated categorisation scheme to given USV data in order to generate symbolic representation transcripts, a modified process is used. It is expected that this new data is preprocessed in the same way as the data used to generate the clustering object, either by applying the provided scripts or repeating the user's alternative preprocessing. Once this is done, an altered feature extraction and segmentation script can be activated.

The feature extraction is identical to that of the clustering generation use case, and it is expected that the same model is used. Feature processing then occurs, following the same sequence of steps as for clustering generation. However, modifications to several scripts mean that the same PCA calculation and internal time-step clustering centroids are applied to this data as was applied to the original data, rather than calculating these again. This ensures the data is truly processed identically to the rest so new segments can be reasonably soft-clustered with the bulk segments. This also accounts for the fact that this use case might be applied to very small amounts of data at a time, which could lead these feature processing steps to overfit or otherwise poorly perform.

With this set of processes, the user has the ability to generate new segments of the same kind and with the same set of features as those used in a generated clustering object. A second script can then be activated, which loads a given clustering object using python unpickling, as well as reading in the given new segments. These segments are then used as an input for the clustering object's soft clustering prediction function, which calculates the probability that each segment falls into each cluster. This full probability matrix may be useful to particularly in-depth research, so an optional argument allows the user to write this to a file for later analysis.

The clustering application script always generates a transcription text file. Each audio clip is listed individually, separated by new lines. For each segment in a given audio clip, its estimated length is given in milliseconds, and the highest-probability cluster assignment is recorded as the most likely category that that segment falls into. That means the user gets a string of numeric codes representing particular categories of call, as well as the segment lengths for analysis. These lengths count small amounts of silence that separate the call signals, but they allow the transcript to be aligned with a spectrogram or other comparison. Several real transcripts generated from USVs experimentally are provided, to show an example of the output format.

7,7,0,1,0,7,7,8,8,6,1,
50,60,40,50,70,70,40,50,50,50,30,

0,7,0,3,
50,60,40,10,

7,7,3,0,7,0,3,0,1,3,
40,50,50,40,40,50,40,60,60,20,

These comma-separated numeric strings of text are simple to read manually, copy into spreadsheets, or import as data into some other analysis tool, accounting for most expected uses. Several examples comparing transcripts to spectrograms of the same section of signal are considered in the evaluation section below. This is the final step of the second use case, and its implementation largely concludes development work on the project.

A handful of related scripts that are not necessary to operate the system normally are also provided, for the instances in which they might be useful. These include a script that modifies wav2vec model checkpoints to address configuration errors (possibly necessary if a new checkpoint of a feature extraction model is used), scripts to cut long USV audio clips into smaller segments (as to train a model, 10s-30s clips are recommended), and scripts that implement a k-means clustering approach to USV segments for any users interested in exploring this approach instead.

6. Experiments and Evaluation

Experimental Tool Use

145 minutes of recording of same-sex pairs of rats during play behaviour were used for this experiment. These were split into 14 ten minute files, and one 5 minute file. During feature processing, three files were rejected due to a GPU error and excluded from final results. These files included both male-male pairs and female-female pairs, but the strain of rat recorded was not available. The intention was to attempt to produce a general scheme using data available, but results might have been better for a scheme specific to one strain and sex, as USV characteristics are known to vary somewhat based on these (Graham et al., 2009; Lenell et al., 2021).

If the recordings were already of condensed USV calls only, on average almost 200,000 calls would fit in this runtime. However, in ~115 minutes of real time, far fewer vocalizations are expected, as rats do not vocalize constantly, and large gaps of silence are consequently expected. After sampling rate adjustments were applied, the effective runtime of the data was roughly 24 hours. Noise reduction was then applied, the effectiveness of which was analyzed by inspecting spectrograms of recorded clips before and after the application of the algorithm.

In clean sections of audio, USVs were isolated effectively (an example of this is visible in figure 11 in the final implementation section). In particularly noisy sections, the noise suppression struggled, and especially loud bumps and spikes in non-USV noise are not masked out by spectral gating. An example of where noise suppression failed to remove a non-USV bump sound is depicted in figure 12.

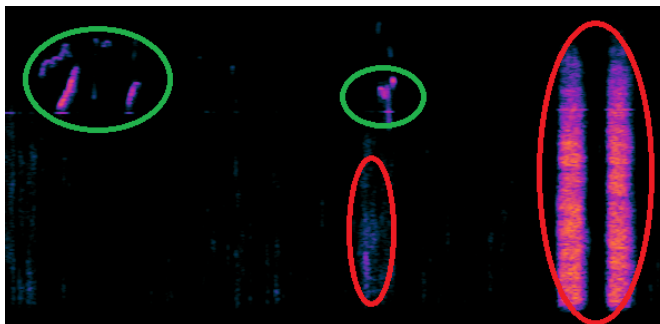


Figure 12: Spectrogram of a noisy USV file after initial noise suppression. Loud spikes of noise remain, circled in red. USV signals are circled in green.

These noise spikes stand out starkly from USVs when comparing spectrographically. That they were not cleaned out during noise suppression represents a partial failure. In the ideal use case, data is clean enough that this is not an issue, but the system needs to be able to function on realistic datasets where microphones are bumped or recording researchers make sounds. Extending noise suppression to automatically identify these bumps would be the first place to make improvements on the system as implemented.

For now, noise suppression effectively removes consistent or quiet background noises, and isolates the relevant frequency band. This is good, but automating noise removal entirely would be preferable. Thankfully, after noise suppression these remaining unwanted sounds are very visually distinct from USVs, so manually cutting them out in a free audio editing software like audacity is trivial if not ideal. Manual removal of noise spikes was performed to refine the data for this experiment.

Subsequently, silence removal was performed to condense the cleaned data into a series of USV calls separated by small gaps of silence, as the wav2vec model expects. This process is the longest part of the entire system by a wide margin. Processing times are discussed in more depth in the quantitative

evaluation subsection below. While slow, the silence removal proved effective, and what was originally nearly two hours of data was trimmed down into only just under a minute of condensed USV calls. With this length, we expect to have somewhere around 1,500 calls in our data.

The feature extraction and processing is significantly faster. It generates 1,811 segments in total, which would mean each segment has an average length of roughly 29 milliseconds. The average length of an affective USV call is typically given as 30-40 milliseconds, but the proportion of different forms of call in our dataset could vary from this average figure. Being within one millisecond of the accepted general average figure (out by only a few percent) is a good indication. These segments are then clustered using the HDBSCAN algorithm, with hyperparameters automatically optimized.

We get as a result an experimental categorisation scheme of *nine* call types, significantly fewer than the fourteen of (Wright et al., 2010). This could be because the system has identified more general superstructures connecting some of the more arbitrary human-defined types. Importantly, however, this is not a definitive answer to conclude studies in this area. Calls vary by strain and sex, our dataset was relatively small, and several processing steps (most particularly noise suppression) could be further refined. So while the system is successful in that it is functional, and able to produce category schemes based on data provided, this experimental result does not represent a genuine categorisation that would be accepted in research. We can now use this experimental scheme in generating a handful of transcripts, however, and visually compare these to spectrograms to closely analyze performance.

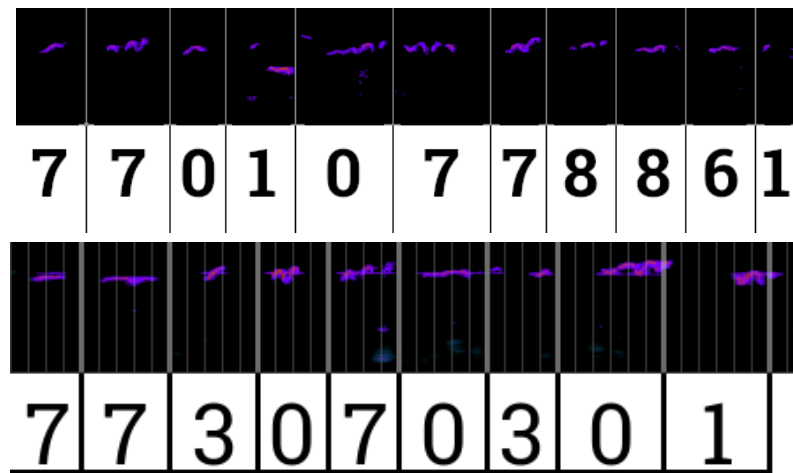


Figure 13: Two examples of USV audio spectrograms compared to their transcribed results with the experimental categorisation.

Considering the segmentation first, it can be seen that our system is actually quite effective at selecting USV calls and isolating them, mostly cleanly matching calls 1-to-1 with our feature representation segments. While an expected outcome, the actual categorisation scheme was judged to be unconvincing in discussions with supervisors, as visual patterns of USV calls within particular call categories are unclear.

This is possibly a result of insufficient data, mixing of calls across sex and strain, noise suppression performing inadequately, or hyperparameters being poorly tuned to the dataset. That is to say, the tool is functional, but its categorisation performance could be significantly improved by more effective use and better data. Additionally, this may be taken as a sign that more refinement is necessary for the automatic processing sections, in order to find better results. The modularity of the implementation makes this easy to accomplish, but does ultimately leave the tool simply experimentally functional, rather than proven in effectiveness over alternatives.

Quantitative/Comparative Evaluation

Generating spectrograms of the right sections of USV data to compare against, and lining them up with transcript segment lengths, is a time-consuming process which presents a practical limitation on evaluation in this way. A small subsample of USV clips were randomly selected to be evaluated, which came to about 2% the size of the bulk dataset used to create the categorisation scheme. This also allowed us to test how well the cluster assignment use case functioned, particularly on individual clips and/or small amounts of data, as might be the case in real use.

In total, just over 77% of segments clearly correlated with a single USV call, entirely and exclusively encapsulated within the bounds of that segment. In 16% of segments, a single USV is clearly selected predominantly, but a small amount of the end of the call signal spills over into the next segment. This is not a disaster, as the receptive field of the convolutional timesteps extends across segment boundaries and all are fully contextualized, so these segments can still be considered reasonably representative. Of course, ideally this spillover should be minimized to limit any spoiling of the mean-pooling of the following segment’s features. Finally, about 6% of cases could be definitively considered genuine segmentation errors, for example splitting particularly long calls into two segments. Figure 14 displays examples of each kind of result more clearly.

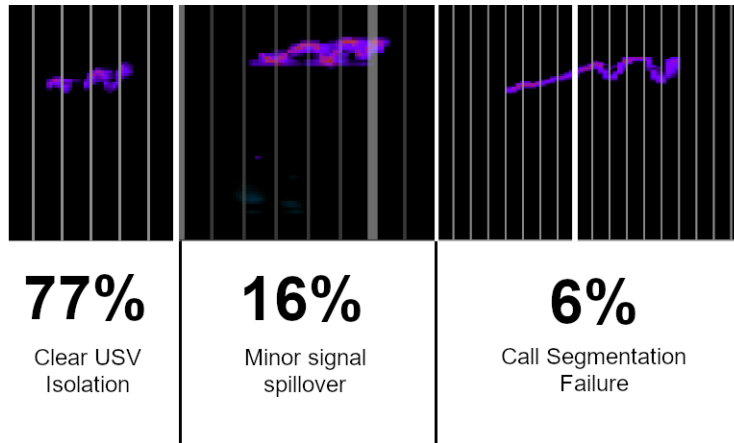


Figure 14: Examples of the results found for call segmentation.

With the caveat that the differences in process could potentially impact the comparison of these values, we can compare to MUPET and DeepSqueak generally in terms of USV detection, by counting our clearly isolated calls and calls with minor spillover at segment boundaries as successes—a 94% rate of success. In testing across multiple tools, (Abbasi et al., 2022) found MUPET had about a 90% rate of true positives (the best equivalent simple “success rate”) on average, and DeepSqueak about 94%. This puts our tool’s performance in the same range as industry standards.

Processing time, which for the sake of this comparison means the time a tool takes to transform an audio file into representations of identified USVs, can also be evaluated. For the avoidance of bias in comparison against our tool, processing speeds for DeepSqueak and MUPET are drawn from other academic sources. (Binder et al., 2022) found that processing using DeepSqueak took an average of ten minutes for two minute files, with significant variance depending on how densely present USVs are in the recording. In contrast, (Van Segbroeck et al., 2017) finds that MUPET takes about thirty seconds to process a six minute file.

Most parts of our preprocessing take negligible time, but several tests showed that our silence removal process takes about 96 seconds for each real minute of recorded data. Following preprocessing, actual processing took about 168 seconds per real minute of recorded data on average, though there is an increase in efficiency as the length of data increases, and as the density of USVs decreases. Including

all preprocessing steps, that means one minute of USV recording takes about 270 seconds to process. Of course, the time-cost of manually sanitizing preprocessed audio should also be fairly considered, as in some cases this may be necessary. This depends greatly on the user, their tools, and their specific data, but a few timed tests using audacity at a casual pace indicate that perhaps 90 seconds on average is appropriate per real minute. Counting this, we get a processing time of 360 seconds per minute.

6 minutes of processing per minute of recording is slightly longer than DeepSqueak and significantly longer than MUPET. This is to be expected, however. Just as DeepSqueak takes significantly longer than MUPET in exchange for additional complexity that addresses criticisms with the prior tool, our tool continues the same evolution. Using wav2vec processes for feature extraction is more computationally expensive than the extraction of manually defined features from spectrograms, but this is necessary to resolve the criticisms of (Goffinet et al., 2021) in that manually defined features may impact results more than we understand or would like. That our tool takes slightly longer than DeepSqueak to function in exchange for removing handpicked feature definition is well worth it, particularly given the difference in processing time is only 20%, when the change from MUPET to DeepSqueak meant a change of 600%. This is also still dramatically faster and more convenient than manual analysis, like that of (Wright et al., 2010).

	USV Detect Success Rate	Processing Time (per minute of audio)	Level of Automation	Feature Definition	Clustering Algorithm
<i>Our Tool</i>	94% (of noise cleaned signal)	6 minutes	Fully Unsupervised (with optional manual refinement)	Automatic	HDBSCAN
<i>DeepSqueak</i>	94%	5 minutes	Fully Unsupervised (with optional manual refinement)	Manual	K-Means
<i>MUPET</i>	91%	5 seconds	Mandatory manual steps	Manual	K-Means

Our tool has comparable rates of USV detection and processing time to the previous industry standard DeepSqueak, while advancing on it in several key areas. Features are no longer handpicked or manually defined, and segments are now clustered using an algorithm that accounts for density differences, noisy data points, and complex cluster shapes in higher dimensions. These changes address a number of previously discussed academic concerns. All core functionalities are implemented, including both the automatic generation of categorisation schemes, and easy symbolic representation of audio files into transcripts of these schemes.

However, automatic noise suppression is not yet fully effective, leaving a gap where manual action is occasionally necessary. Our processing time, while acceptable, could also be significantly optimized if faster techniques for silence removal were adopted. Our USV detection success rate, while within industry standards, is imperfect and would benefit from refinement in the segmentation process. Finally, our experimental categorisation scheme was ultimately unimpressive, and while our dataset can serve as a partial explanation, this is still indicative that further improvements could be made.

Ultimately, this tool has been successfully designed and implemented in full, and makes key improvements on past solutions in order to address new concerns in the field. But it is not the end of the road, and there is still room for improvements and new development before it could be used confidently to answer major questions about the structure of rat communication.

7. Conclusions and Future Work

This project has successfully implemented a solution that in some way accomplishes all primary design requirements. Completely automatically and without supervision, raw USV audio can be processed all the way into new call categorisation schemes, and/or transcribed into a scheme generated in this way. There are points where the user may choose to intervene, for example in further cleaning data of noise or tweaking algorithm hyperparameters, but these steps are not strictly necessary, meaning there is a fully functional pipeline from start to finish, without human decision-making.

However, this implementation is imperfect, and this is not the end of the journey for USV analysis automation. Thankfully, this tool was designed as a series of easily detached parts, separated scripts all called from a handful of master scripts activated by the user. This makes swapping out particular systems or making changes internally to produce better results fairly simple.

The most significant area of potential improvement is in automatic noise suppression. The current implementation works well for constant background noise and quiet non-USV sounds, but particularly loud noise can pass through the spectral gating step. If this could be prevented, this would allow the noise suppression to be confidently run fully automatically even on quite noisy data, which would both help the goal of effective end-to-end unsupervised processing, as well as save time. Potential avenues to explore in this direction include training a noise detector neural network similar to the one used in DeepSqueak, or designing an algorithm that cuts out sounds with too broad a frequency range.

Another potential keystone improvement would be training a domain-specific feature extraction model, rather than using the generic XLSR-53. While this model has been helpful, and integrating off-the-shelf models served to ease implementation and make the tool more accessible, a wav2vec2.0 model trained exclusively on USV calls could be expected to result in more relevant feature outputs, which could improve segment clustering results.

The challenge in training a new feature extraction model is obtaining enough USV data to train a model from scratch, as fine-tuning an existing model is impossible due to the requirement for labeled data. Additionally, as discussed USVs vary by strain and sex, so training a general USV model would need plenty of data from across a wide spectrum of rats, or different models could be trained for different strains and/or sexes. This could produce the best results, in theory, but would be very time-consuming, and the user would need to have similar data (or be able to train their own locally).

There are also several areas where the system is perfectly functional but usability could be improved. Finding an alternative silence removal algorithm that functions faster would shrink processing times, and developing a simple GUI could make the tool easier to use than the command line for some users. A GUI would also make it possible to display spectrograms with their transcripts within the tool itself, which would make visual inspection of categorisation schemes easier.

Finally, there are a few challenging directions a project could be taken in to build off of this tool and discover interesting new insights. Creating a model “language” similar in structure to USVs with defined correct categorisations would let this tool (and others) be analyzed for their performance against known data labels. Alternatively, developing a system to reconstruct audio signals from their wav2vec representations could allow us to start working in the other direction—a text-to-speech to go with our speech-to-text. Archetypical call signals for different categories of USV call could be synthesized in a given order, allowing us to construct messages in a given categorisation scheme, both to evaluate their own veracity and to experiment with their behavioural impacts on rats.

Bibliography

- Abbasi R, Balazs P, Marconi MA, Nicolakis D, Zala SM, Penn DJ. (2022). Capturing the songs of mice with an improved detection and classification method for ultrasonic vocalizations (BootSnap). *PLoS Comput Biol* 18(5): e1010049. <https://doi.org/10.1371/journal.pcbi.1010049>
- Aharonson V, Aharonson E, Raichlin-Levi K, Sotzianu A, Amir O, Ovadia-Bleckman Z. (2017). A real-time phoneme counting algorithm and application for speech rate monitoring, *Journal of Fluency Disorders*, Vol. 51: 60-68. doi: 10.1016/j.jfludis.2017.01.001
- Albawi S, Mohammed T, Al-Zawi S. (2017). Understanding of a convolutional neural network, 2017 International Conference on Engineering and Technology (ICET), 2017, pp. 1-6, doi: 10.1109/ICEngTechnol.2017.8308186.
- Baevski A, Zhou H, Mohamed A, & Auli M. (2020). wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations, arXiv. <https://arxiv.org/abs/2006.11477>
- Baevski A, Conneau A, Hsu W & Auli M. Unsupervised Speech Recognition, arXiv (2021). <https://arxiv.org/abs/2105.11084>
- Chong B. (2021) K-means clustering algorithm: a brief review. *Academic Journal of Computing & Information Science*, Vol. 4, Issue 5: 37-40. <https://doi.org/10.25236/AJCIS.2021.040506>.
- Binder M, Pranske Z, Lugo J. (2022). The Deepsqueak analysis system is as accurate, yet more efficient, than the Avisoft system across C57BL/6, FVB.129, and FVB mice, *Brain Disorders*, Volume 8, 2022, 100055, ISSN 2666-4593, <https://doi.org/10.1016/j.dscb.2022.100055>
- Bommasani, Rishi & Hudson, Drew & Adeli, Ehsan & Altman, Russ & Arora, Simran & Arx, Sydney & Bernstein, Michael & Bohg, Jeannette & Bosselut, Antoine & Brunskill, Emma & Brynjolfsson, Erik & Buch, Shyamal & Card, Dallas & Castellon, Rodrigo & Chatterji, Niladri & Chen, Annie & Creel, Kathleen & Davis, Jared & Demszky, Dora & Liang, Percy. (2021). On the Opportunities and Risks of Foundation Models.
- Brudzynski S. (2009). Communication of Adult Rats by Ultrasonic Vocalization: Biological, Sociobiological, and Neuroscience Approaches, *ILAR Journal*, Volume 50, Issue 1, 2009, Pages 43–50, <https://doi.org/10.1093/ilar.50.1.43>
- Clustering. (30th September 2022). Hipparchus. <https://www.hipparchus.org/hipparchus-clustering/>
- Coffey, K.R., Marx, R.G. & Neumaier, J.F. (2019). DeepSqueak: a deep learning-based system for detection and analysis of ultrasonic vocalizations. *Neuropsychopharmacol.* 44, 859–868. <https://doi.org/10.1038/s41386-018-0303-6>
- Goffinet, J., Brudner, S., Mooney, R., & Pearson, J. (2021). Author response: Low-dimensional learned feature spaces quantify individual and group differences in vocal repertoires. <https://doi.org/10.7554/elife.67855.sa2>
- Graham, L. K., Yoon, T., Lee, H. J., & Kim, J. J. (2009). Strain and sex differences in fear conditioning: 22 kHz ultrasonic vocalizations and freezing in rats. *Psychology & Neuroscience*, 2(2), 219–225. <https://doi.org/10.3922/j.psns.2009.2.015>
- Holy TE, Guo Z. (2005). Ultrasonic songs of male mice. *PLoS Biol* 3:e386

- Kaltwasser MT. (1990). Acoustic signaling in the black rat (*Rattus rattus*). *J Comp Psychol* 104:227–232
- Kershenbaum A, Sayigh LS, Janik VM. (2013). The Encoding of Individual Identity in Dolphin Signature Whistles: How Much Information Is Needed?. *PLOS ONE* 8(10): e77671. <https://doi.org/10.1371/journal.pone.0077671>
- Lee B. (1950). "Effects of Delayed Speech Feedback", *The Journal of the Acoustical Society of America* 22, 824-826 <https://doi.org/10.1121/1.1906696>
- Lee H & Grosse R & Ranganath R & Ng A. (2009). Convolutional deep belief networks for scalable unsupervised learning of hierarchical representations. *Proceedings of the 26th International Conference On Machine Learning, ICML 2009*. 77. 10.1145/1553374.1553453.
- Lee C, O'Donnell T, Glass J. (2015) "Unsupervised Lexicon Discovery from Acoustic Input." *Transactions of the Association for Computational Linguistics, Volume 3*. 2015 Association for Computational Linguistics
- Lenell C, Broadfoot CK, Schaen-Heacock NE, Ciucci MR. (2021). Biological and Acoustic Sex Differences in Rat Ultrasonic Vocalization. *Brain Sci.* 2021 Apr 4;11(4):459. doi: 10.3390/brainsci11040459. PMID: 33916537; PMCID: PMC8067311.
- McInnes L, Healy J, Astels S. (2017). hdbscan: Hierarchical density based clustering, *Journal of Open Source Software*, 2(11), 205, doi:10.21105/joss.00205
- Mines MA, Hanson BF, Shoup JE. (1978). Frequency of Occurrence of Phonemes in Conversational English. *Language and Speech*. (1978);21(3):221-241. doi:10.1177/002383097802100302
- Mittal N, Thakore N, Reno J, Bell R, Maddox W, Schallert T, Duvauchelle C. (2018). Alcohol-naïve USVs distinguish male HAD-1 from LAD-1 rat strains, *Alcohol*, Volume 68, 2018, Pages 9-17, ISSN 0741-8329, <https://doi.org/10.1016/j.alcohol.2017.09.003>.
- Moulavi D, Jaskowiak P, Campello R, Zimek A, Sander J (2014), Density-Based Clustering Validation, In *Proc. of 14th SIAM International Conference on Data Mining*, doi:10.1137/1.9781611973440.96
- Sainburg T. (2019). timsainb/noisereducer: v1.0 (db94fe2). Zenodo. <https://doi.org/10.5281/zenodo.3243139>
- Sales GD. (1972). Ultrasound and mating behaviour in rodents with some observations on other behavioural situations. *J Zool London* 168:149–164
- Scattoni ML, Gandhi SU, Ricceri L, Crawley JN. (2008). Unusual repertoire of vocalizations in the BTBR T+tf/J mouse model of autism. *PLoS ONE* 3:e3067
- Schneider, Steffen & Baevski, Alexei & Collobert, Ronan & Auli, Michael. (2019). wav2vec: Unsupervised Pre-Training for Speech Recognition. 3465-3469. 10.21437/Interspeech.2019-1873.
- Seidisarouei M, van Gorp S, Pranic N, Calabus I, van Wingerden M, Kalenscher T. (2021). Distinct Profiles of 50 kHz Vocalizations Differentiate Between Social Versus Non-social Reward Approach and Consumption. *Front. Behav. Neurosci.*, 21 June 2021, Sec. Motivation and Reward, <https://doi.org/10.3389/fnbeh.2021.693698>

- Simola N. (2015). Rat Ultrasonic Vocalizations and Behavioral Neuropharmacology: From the Screening of Drugs to the Study of Disease. *Current Neuropharmacology*, Volume 13, Number 2, 2015, pages 164-179, <https://doi.org/10.2174/1570159x13999150318113800>
- Stewart M. (2019). Simple Introduction to Convolutional Neural Networks. Available online: <https://towardsdatascience.com/simple-introduction-to-convolutional-neural-networks-cdf8d3077bac> (accessed October 8th 2022)
- Sungur, A. Ö., Schwarting, R. K., & Wöhr, M. (2015). Early communication deficits in the Shank1 knockout mouse model for autism spectrum disorder: Developmental aspects and effects of social context. *Autism Research*, 9(6), 696-709. <https://doi.org/10.1002/aur.1564>
- Takahashi N, Kashino M, Hironaka N (2010) Structure of Rat Ultrasonic Vocalizations and Its Relevance to Behavior. *PLoS ONE* 5(11): e14115. <https://doi.org/10.1371/journal.pone.0014115>
- Thompson B, Leonard K, Brudzynski S. (2006). Amphetamine-induced 50 kHz calls from rat nucleus accumbens: A quantitative mapping study and acoustic analysis. *Behavioural Brain Research* Volume 168, Issue 1, 15 March 2006, Pages 64-73. <https://doi.org/10.1016/j.bbr.2005.10.012>
- Van Segbroeck M, Knoll AT, Levitt P, Narayanan S. (2017). MUPET-Mouse Ultrasonic Profile ExTraction: A Signal Processing Tool for Rapid and Unsupervised Analysis of Ultrasonic Vocalizations. *Neuron*. 2017 May 3;94(3):465-485.e5. doi: 10.1016/j.neuron.2017.04.005. PMID: 28472651; PMCID: PMC5939957.
- Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez A, Kaiser L, Polosukhin I (2017) Attention Is All You Need. In *Proc. of NIPS*, 2017. <https://doi.org/10.48550/arxiv.1706.03762>
- Vivian JA, Miczek KA. (1993). Morphine attenuates ultrasonic vocalization during agonistic encounters in adult male rats. *Psychopharmacology (Berl)* 111:367–375
- White NR, Cagiano R, Moises AU, Barfield RJ. (1990). Changes in mating vocalizations over the ejaculatory series in rats (*Rattus norvegicus*). *J Comp Psychol* 104:255–262
- Wright, J.M., Gourdon, J.C. & Clarke, P.B.S. (2010). Identification of multiple call categories within the rich repertoire of adult rat 50-kHz ultrasonic vocalizations: effects of amphetamine and social context. *Psychopharmacology* 211, 1–13. <https://doi.org/10.1007/s00213-010-1859-y>
- Zhang, Jiachi & Liu, Liu & Fan, Yuanyuan & Zhuang, Lingfan & Zhou, Tao & Piao, Zheyang. (2020). Wireless Channel Propagation Scenarios Identification: A Perspective of Machine Learning. *IEEE Access*. PP. 1-1. 10.1109/ACCESS.2020.2979220.